

Reporte Técnico: Clasificación de Tráfico Urbano mediante Redes Neuronales Artificiales desde Cero

José Daniel Morales Ocampo

Maestría en Inteligencia Artificial Plus Aplicada al Desarrollo Regional
Algoritmos de Programación Avanzada

11 de junio de 2026

Resumen

Este reporte describe la reestructuración y el diseño de arquitectura orientada a objetos (POO) para el proyecto final de clasificación del tráfico urbano. Traduciendo la base conceptual matemática y algorítmica de la práctica 3, se implementó una red neuronal feedforward completamente desde cero usando únicamente la biblioteca NumPy. El sistema incluye propagación hacia adelante multidimensional (procesamiento en lotes o batches), retropropagación analítica, validación numérica de gradientes mediante diferencias centrales, búsqueda sistemática de hiperparámetros con exportación de bitácoras JSON y un modo de inferencia directa para producción. Los resultados experimentales muestran un modelo robusto capaz de alcanzar una exactitud del 100 % en validación y del 92.6 % en el conjunto de prueba independiente.

Introducción

El problema del tráfico urbano involucra la clasificación automática del estado de vialidades críticas bajo cuatro categorías potenciales: *normal*, *critical*, *warning* y *desconocido*. Dadas las variables numéricas de flujo vehicular (*vehiculos_hora*, *velocidad_promedio_kmh*, *densidad_vehicular* y *tiempo_espera_s*), este reporte documenta una arquitectura modular reutilizable que implementa el aprendizaje automático profundo partiendo de los primeros principios de optimización lineal y no lineal.

Estructura del Proyecto y Explicación de Módulos

El código fuente del sistema se distribuye en una jerarquía de directorios estructurada para separar las responsabilidades de datos, lógica central y salidas de ejecución.

La carpeta **Core/** representa el núcleo matemático del sistema y contiene los módulos de lógica central. Dentro de este directorio, la clase **Sanitizer** en el archivo **Sanitizer.py** se encarga de limpiar los datos crudos leídos del archivo CSV de entrada, removiendo espacios en blanco sobrantes, convirtiendo valores vacíos en nulos, deduplicando llaves y formateando columnas numéricas con una función personalizada de parseo de comas decimales. La clase **Builder** en el archivo **Builder.py** toma las muestras sanitizadas y construye el dataset formal; se ocupa de validar la homogeneidad de la longitud de las características de entrada, normalizar etiquetas inconsistentes, traducir los metadatos relevantes y realizar la división del dataset en los subconjuntos correspondientes. La clase **Query** en el archivo **Query.py** facilita búsquedas rápidas por identificador único, filtra colecciones por etiquetas de tráfico o metadatos de cruce, y provee estadísticas descriptivas de los descriptores de tráfico. La clase **Layer** en el archivo **Layer.py** abstrae una capa completamente conectada de la red neuronal, encapsulando sus correspondientes matrices de pesos y sesgos junto con la verificación formal de consistencia.

de dimensiones internas. La clase **Prediction** en el archivo **Prediction.py** gestiona el flujo secuencial de la inferencia, encadenando las instancias de capa cargadas en su memoria interna para predecir las distribuciones de probabilidad y las clases discretas asociadas a cada muestra de tráfico evaluada. La clase **Trainer** en el archivo **Trainer.py** reúne la maquinaria del aprendizaje automático, implementando la propagación y retropropagación de gradientes, la métrica de entropía cruzada, la rutina de chequeo numérico, la bitácora de entrenamiento y la exportación de curvas de aprendizaje.

Adicionalmente, la carpeta **Datasets/** actúa como repositorio de almacenamiento para los archivos CSV crudos que registran los flujos históricos de tráfico urbano. La carpeta **Exports/** funciona como la salida consolidada del sistema, guardando los datasets estructurados en formato JSON, el modelo entrenado serializado en formato NPZ, las bitácoras consolidadas de búsqueda hiperparamétrica y los gráficos generados en formato PNG. El archivo raíz **main.py** coordina la secuencia de procesamiento como punto de entrada de la aplicación, mientras que el archivo **requirements.txt** describe las bibliotecas externas mínimas necesarias para ejecutar el sistema.

Metodología de Preparación de Datos

La ingesta y adecuación de datos se coordina mediante la llamada secuencial de las herramientas del núcleo. En primer lugar, la lectura del archivo CSV produce una representación cruda en diccionarios de python que es procesada por el desinfectador. Una vez removidas las inconsistencias de formateo y los duplicados, el constructor de datasets consolida una matriz numérica de características y un vector entero de etiquetas alineadas. En este paso, el constructor normaliza los valores textuales de las etiquetas mapeando cadenas truncadas a su representación formal y asignando índices numéricos. Asimismo, los subconjuntos de datos se dividen de manera desordenada y aleatoria para construir conjuntos disjuntos de entrenamiento, validación y prueba, guardando cada uno de ellos en archivos independientes JSON dentro de la carpeta de exportaciones para auditorías externas.

Formulación Matemática de la Capa Densa y Activaciones

La clase **Layer** representa una capa densa y completamente conectada. A diferencia del diseño original restrictivo a vectores 1D, se reestructuró la validación y el cálculo algebraico para operar con matrices multidimensionales de cualquier rango de lotes (batches 2D y superiores) empleando la multiplicación matricial en el último eje:

$$Z = \text{np.matmul}(X, W) + b \quad (1)$$

Donde X representa la matriz de entrada, W la matriz de pesos, b el vector de sesgos, y Z la entrada neta antes de la activación.

Para la capa oculta, se utiliza la activación de Rectificador Lineal (ReLU), la cual introduce no-linealidades en la red y se evalúa como:

$$f(z) = \max(0, z) \quad (2)$$

Su derivada analítica, empleada durante el paso de retropropagación del error, se define como:

$$f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases} \quad (3)$$

Para la capa final de salida, se aplica la función Softmax para transformar las salidas netas lineales (logits) en un vector que represente una distribución de probabilidad de las clases. Su

cálculo incorpora un factor de escala para mantener estabilidad numérica ante valores extremos:

$$S(Z)_{ij} = \frac{e^{Z_{ij} - \max(Z_i)}}{\sum_k e^{Z_{ik} - \max(Z_i)}} \quad (4)$$

La exactitud de la red se cuantifica utilizando la pérdida de entropía cruzada mutua para problemas de clasificación multiclase:

$$L = -\frac{1}{N} \sum_{i=1}^N \log(P_{i,y_i} + 10^{-9}) \quad (5)$$

Donde N es el tamaño del lote, y_i es el índice de clase objetivo de la muestra i , y P_i representa el vector de probabilidad resultante del paso Softmax.

Algoritmo de Retropropagación

El entrenamiento de los pesos y sesgos se basa en el cálculo de las derivadas parciales de la pérdida con respecto a cada parámetro del sistema en reversa.

En la capa de salida (Capa 2), el gradiente acumulado respecto a la entrada neta se simplifica al restar el indicador unitario de la clase correcta a las probabilidades estimadas por Softmax y promediar sobre el tamaño del lote, lo cual produce el tensor de error $dZ^{[2]}$. A partir de este error, el gradiente para los pesos del segundo nivel se calcula como el producto matricial de las activaciones transpuestas de la capa oculta por el error de salida, resultando en $dW^{[2]}$. El gradiente para los sesgos de salida se obtiene al colapsar el error $dZ^{[2]}$ sumando a lo largo del eje del lote, denotado como $db^{[2]}$.

Para propagar el error hacia la capa oculta (Capa 1), se multiplica el error de salida $dZ^{[2]}$ por la matriz transpuesta de pesos $W^{[2]}$, produciendo el error respecto a las activaciones intermedias $dA^{[1]}$. Posteriormente, este vector se multiplica de forma elemental por la derivada de la activación ReLU evaluada sobre las entradas netas de la primera capa, dando el error neto de la capa oculta $dZ^{[1]}$. En última instancia, los gradientes de la primera capa se obtienen de forma análoga multiplicando la matriz de entradas original transpuesta por el error intermedio para dar $dW^{[1]}$, y sumando el error intermedio sobre las muestras para dar $db^{[1]}$.

Una vez calculados los gradientes analíticos, el optimizador actualiza los parámetros restando el producto de la tasa de aprendizaje por el respectivo tensor de derivadas parciales.

Verificación y Gradient Checking

Para asegurar la exactitud de la formulación analítica de la retropropagación, el entrenador incorpora un paso de verificación numérica (*Gradient Checking*) que aproxima la derivada mediante diferencias centrales:

$$\frac{\partial L}{\partial W_{ij}} \approx \frac{L(W_{ij} + \epsilon) - L(W_{ij} - \epsilon)}{2\epsilon} \quad (6)$$

Donde se emplea una perturbación controlada de $\epsilon = 10^{-7}$.

La comparación en el primer peso de la red ($W_{0,0}^{[1]}$) demostró un acoplamiento casi perfecto entre la aproximación analítica y numérica:

- Gradiente Numérico: 0,03080770
- Gradiente Analítico: 0,03080770
- Diferencia Absoluta: $3,18605283 \times 10^{-9}$

Al ser el valor menor que 10^{-7} , se valida matemáticamente la correctitud del algoritmo **backward**.

Resultados Experimentales y Búsqueda de Hiperparámetros

El entrenamiento de la red neuronal base (8 neuronas ocultas, learning rate $\eta = 0,03$ y 600 épocas) alcanzó una exactitud del 100 % en el conjunto de entrenamiento, del 92.0 % en validación y del 92.6 % en el conjunto de prueba independiente. La matriz de confusión evaluada en las 27 muestras del set de prueba independiente se presenta en la Tabla 1:

Cuadro 1: Matriz de Confusión de Clasificación de Tráfico Urbano				
Real \ Predicha	Critical	Desconocido	Normal	Warning
Critical	12	0	0	1
Desconocido	0	0	1	0
Normal	0	0	6	0
Warning	0	0	0	7

De acuerdo con las directrices de la Actividad 6, se ejecutó una búsqueda sistemática evaluando cuatro configuraciones de hiperparámetros. Los resultados fueron almacenados de manera organizada en el archivo `Exports/results.json`, cuyos detalles experimentales se resumen en la Tabla 2:

Cuadro 2: Resultados del Laboratorio de Experimentos de Hiperparámetros					
Exp.	Neuronas Ocultas	Learning Rate	Épocas	Exactitud Val.	Pérdida Val.
0	8	0.003	600	76.0 %	0.5900
1	8	0.030	600	100.0 %	0.1094
2	8	0.300	600	100.0 %	0.0108
3	16	0.030	600	100.0 %	0.0699

Las curvas específicas de pérdida y exactitud para cada uno de los experimentos se guardaron de manera automatizada bajo el formato de nombres `modelo-00X-loss_curve.png` y `modelo-00X-accuracy_curve.png` en la carpeta de exportaciones.

Modos de Ejecución

El script de entrada `main.py` ofrece dos modos operativos complementarios:

El primer modo corresponde al entrenamiento completo y búsqueda hiperparamétrica. Limpia los datos crudos, inicializa y entrena la red base, ejecuta la verificación numérica de gradientes, ejecuta las configuraciones de experimentos guardando las curvas de aprendizaje individuales en la carpeta `Exports/`, genera el archivo resumido de bitácoras `results.json` y serializa los pesos finales en `modelo_entrenado.npz`. Este modo se ejecuta llamando directamente al script principal en la terminal:

```
1 python main.py
```

El segundo modo corresponde a la inferencia directa en producción. Permite utilizar directamente un modelo entrenado cargado de disco, evitando repetir la fase de cómputo de gradientes para agilizar el procesamiento. Este comando lee la ruta del modelo serializado, normaliza las características del set de prueba independiente con los parámetros cargados de la sesión de entrenamiento (media y desviación estándar) y realiza inferencias directas imprimiendo las métricas asociadas:

```
1 python main.py -m ./Exports/modelo_entrenado.npz
```

Conclusión

La reestructuración modular de la red neuronal en clases cohesivas proporciona una solución de software altamente reutilizable para problemas de clasificación linealmente no separables. Al desacoplar la validación de lotes, el cálculo del pase backward y la inferencia directa, el código final satisface las mejores prácticas en el desarrollo de software científico y proporciona un modelo funcional altamente exacto para el análisis de tráfico urbano.